pfi

Python for Fixed Income

COURSE 10 · IMPLEMENTING RAG

LLM Judges and Their Limits

WEEK 3 · DAY 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: a model judges answers, and a human judges the judge

45 saved answers, 90 saved verdicts

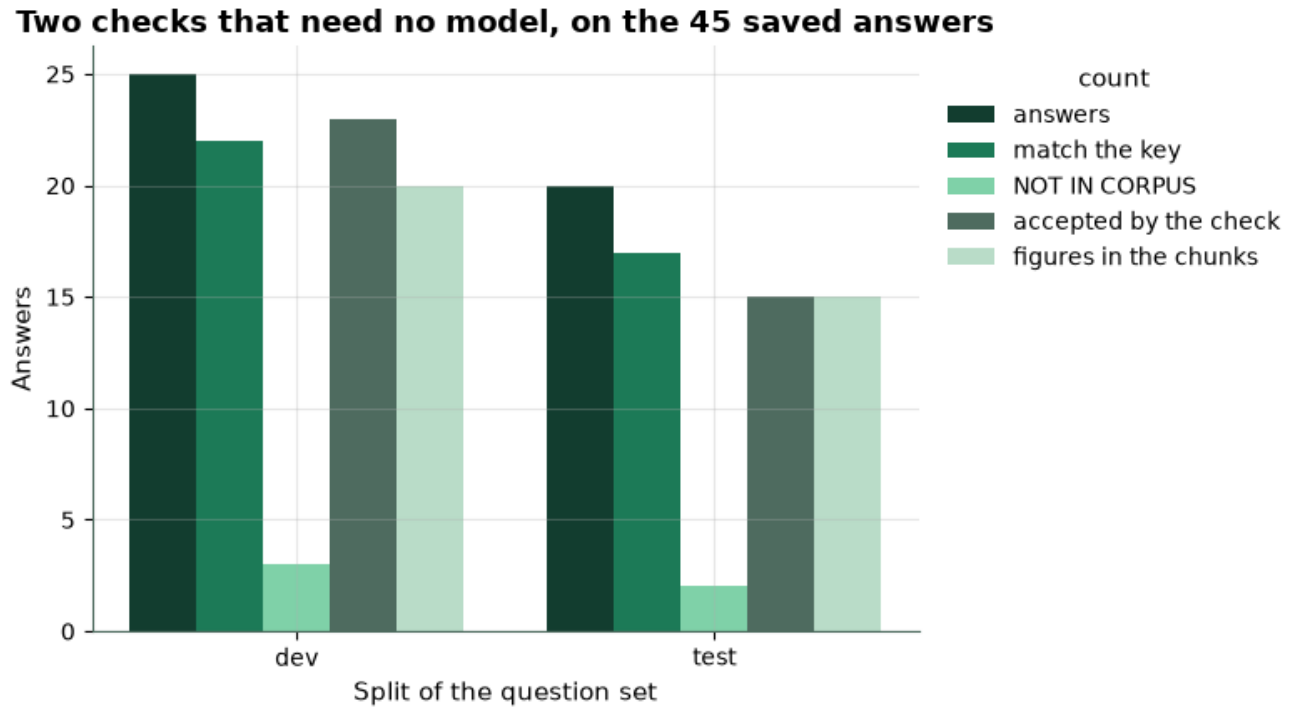
Answers	45: 25 development, 20 test
Keys	41 in the 10-K; 4 it cannot answer
Setting	day 8's rerank, from saved files
Judges	groundedness, relevance: 2 per answer
Torch	none today; no model on your machine

Today

1. Three ways to check an answer
2. The groundedness judge
3. The relevance judge
4. Both judges on every answer
5. The human labels
6. Agreement and Cohen's kappa
7. Where they part
8. A model judging its own answers
9. A verdict is a measurement

A verdict is a label a model returned, never the truth; the key is never in a judge's request.

Two checks that need no model



On 45 answers

39
match their key (key grader)

5 (4 right)
reply NOT IN CORPUS

38
accepted by check_citations

35 of 41
key's figures in the chunks sent

By split: dev 25 answers, 22 matched, 23 accepted; test 20 answers, 17 matched, 15 accepted. The five no-match answers are all retrieval misses.

A judge is a request with three things fixed

Fixed in every judge request

Instruction
GROUNDED_PROMPT or RELEVANT_PROMPT, the only system message

Format
VERDICT_FORMAT: a strict JSON schema, two fields, three labels

Record
Verdict: verdict a Literal of yes, partly, no; reason text

Sampling
temperature 0, seed 42, max_tokens 200

What each judge is given

Groundedness
the question, the chunks labelled as rag_prompt labels them, the answer

Relevance
the question and the answer; no chunks

Neither
the answer key, ever (the AI check plants that bug)

Order
the instruction, then the user message; nothing in front of them

Allowed names

the judge's verdict

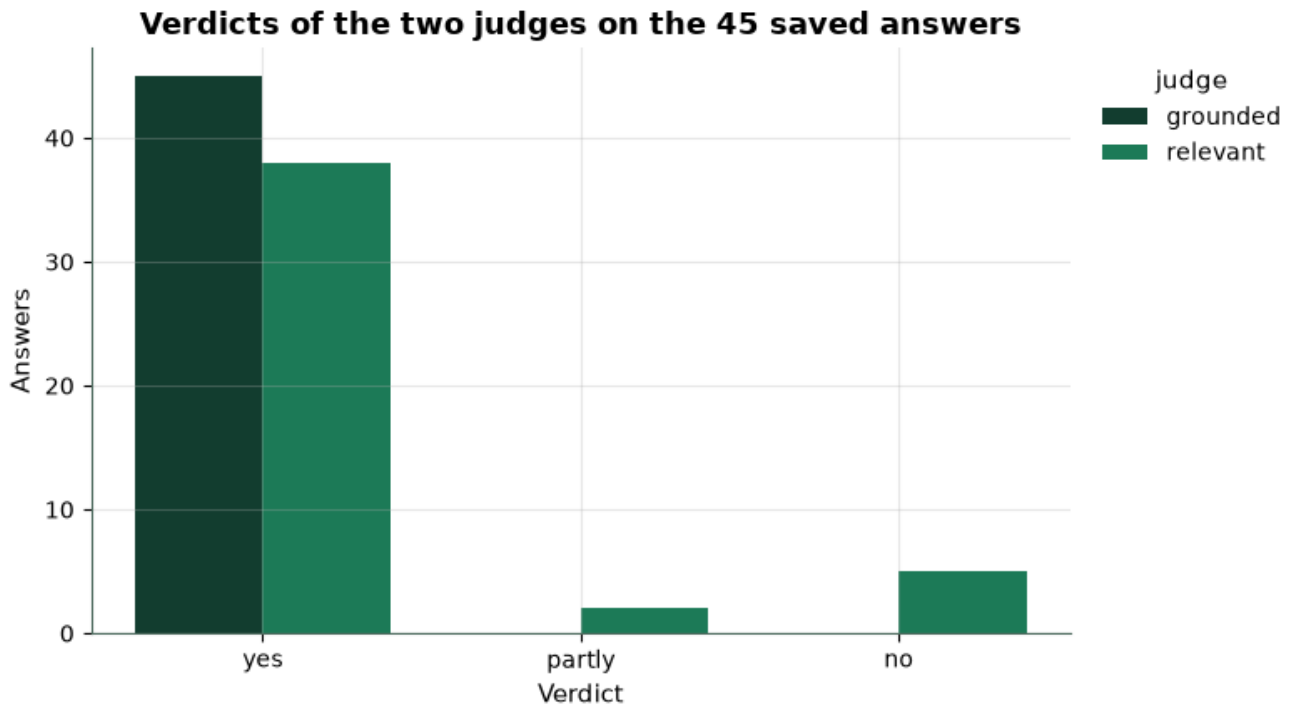
groundedness as the judge scored it

Never called

the truth

accuracy, unless against a key

Ninety verdicts, two per answer



Counts

	yes	partly	no
grounded	45	0	0
relevance	38	2	5

key grader matched 39; check accepted 38

90 requests: 37,841 prompt and 2,817 completion tokens, replayed from the saved run.

Every answer was judged grounded: the judge's yes count sits above the key's match count.

The relevance judge's no, and what its request holds

Relevance: no and partly

5
answers marked no

5
of them NOT IN CORPUS replies (4 matched by the key: NOT IN CORPUS is their key)

2
marked partly: test08, test17, half answers

0 of 5
reasons that speak of a document, a text, a quote, or a citation it was never given

The relevance judge's only system message

```
You check whether an answer addresses its question. Reply with
JSON only: verdict "yes" if it answers what was asked, "p...
```

The first recording put Course 7's extraction prompt in front of it: given no document, the judge marked right answers no for reasons about a document it never saw.

The helper now sends the instruction and the user message, nothing else, and the verdicts were recorded again.

A judge's verdict depends on everything in its request, the system message included.

The relevance instruction says nothing about NOT IN CORPUS; the groundedness instruction does. Two instructions, two readings of the same five replies.

A judge measured only by another model measures nothing

The workbook

1. One row per saved answer, in a shuffled order
2. The question, the three chunks with their citation labels, the answer
3. Two label columns restricted to yes, partly, no; a note column
4. One page of instructions: chunks for groundedness, question for relevance
5. No judge verdict and no answer key shown; a reviewer of the course labelled first

course10_human_labels.csv

`answer_key: the saved file's name`

`labeller`

`grounded: yes, partly, no`

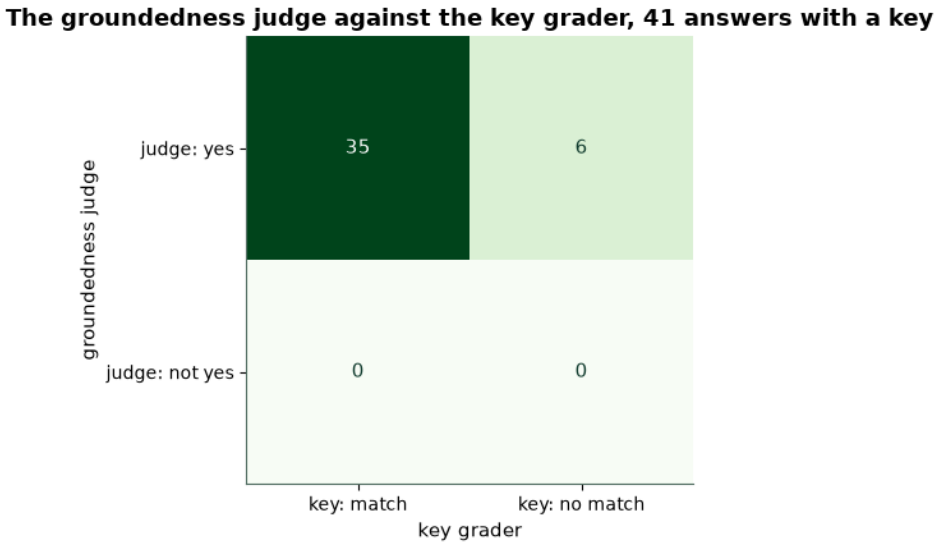
`relevant: yes, partly, no`

`note`

human labels: 45 answers, labelled by course reviewer; grounded yes 42, partly 3, no 0; relevant yes 38, partly 7, no 0

Judge against the reviewer: groundedness kappa 0.0000 (the judge's column has one label); relevance kappa 0.2063, the same label on 35 of 45 (they part on the 5 NOT IN CORPUS replies and on 5 answers the reviewer marked partly).

The groundedness judge against the key grader, 41 answers



kappa = (po - pe) / (1 - pe)

```
35  6
```

judge yes: key match, key no match

```
0  0
```

judge not yes: key match, key no match

```
0.8537
```

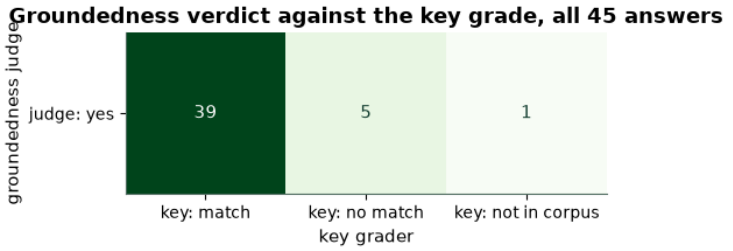
po, the share where the two agree

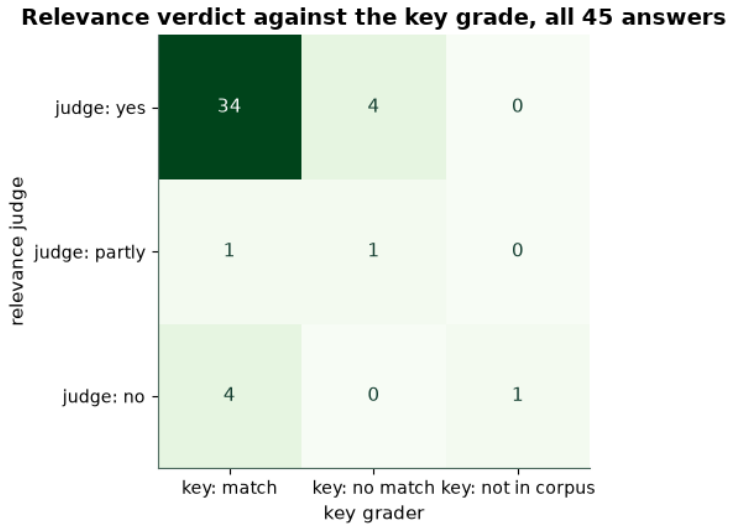
```
0.0000
```

Cohen's kappa: one label in the judge's column, so po equals pe

Both columns must use the same words: cohen_kappa compares strings, and yes against match returns 0 without a word. The key grader's labels are renamed first.

Three labels against three grades, and the six that part





Judge yes, key not matched

`test08, test07, dev23, dev22, test01, dev19`

6 answers, 1 a NOT IN CORPUS reply; the other direction: 0

groundedness yes: 39 match, 5 no match, 1 not in corpus; relevance yes / partly / no by grade: 34, 4, 0 / 1, 1, 0 / 4, 0, 1

dev22 infers total assets from a percentage and cites nothing; dev23 gives Item 7's interest expense where the segment note was asked. The check and the judge accept dev23; the key and a reader with the page do not. A judge given the chunk and the answer cannot tell a right line from a wrong one in the same table.

What the key sees and the judge does not

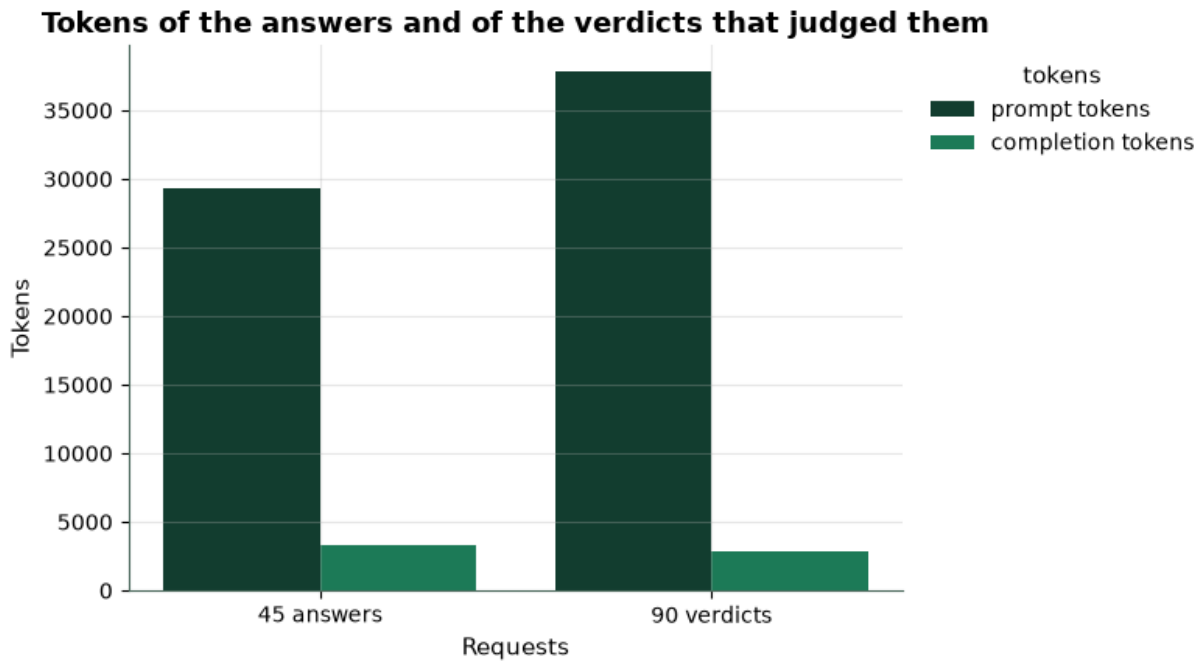
The 5 answers the key grader marks no match

id	grade	figures in chunks	check	groundedness	relevance
test08	no match	no	no	yes	partly
test07	no match	no	yes	yes	yes
dev23	no match	no	yes	yes	yes
dev22	no match	no	no	yes	yes
test01	no match	no	no	yes	yes

The limit, measured

The groundedness judge says yes to 5 of the 5; the key's figures were in the chunks sent for 0: every miss is a retrieval miss, and the model answered from what it had (a nearby table, the other year's column, a share count read as a head count, an inference from a percentage). Grounded is a property of the answer and its chunks; right is a property of the answer and the document. The relevance judge marked 4 of the 5 yes: on the point and from the wrong line is relevant, and it was told not to check whether the answer is true. The T4 cell runs the local model as a second judge, in Colab, on the student's own T4.

Two error rates, and what the verdicts cost



At the list price in ragkit.PRICES

45 answers: 45 requests

29,277 prompt and 3,282 completion tokens; 0.016962 dollars

90 verdicts: 90 requests

37,841 prompt and 2,817 completion tokens; 0.019644 dollars

judges: 1.16 times the answers' cost

A yes is worth the share of yes verdicts a human also marked yes, and catches the share of human yes labels it also marked yes: two counts on 45 answers, one answer is 2 points.

Excel side by side, and Git

Excel side by side (tested in Excel)

`=COUNTIFS($E$2:$E$46,$K2,$C$2:$C$46,L$1)`	groundedness 3 by 3: 39, 5, 1, 0, 0, 0, 0, 0, 0, equal to Python
`=COUNTIFS($F$2:$F$46,$P2,$C$2:$C$46,Q$1)`	relevance 3 by 3: 34, 4, 0, 1, 1, 0, 4, 0, 1, equal to Python
`=COUNTIF($I$2:$I$46,"yes")`	answers with a key: 41
`=(L7-L8)/(1-L8)`	kappa 0.0000 (relevance 0.3844), equal to cohen_kappa
`=(D1-D2)/(1-D2)`	#DIV/0! when both columns use one label; cohen_kappa raises
`Conditional Formatting, Color Scales`	1 format condition on the nine cells: Excel's heatmap

po (L7) and pe (L8), written out: 0.8537 and 0.8537, equal to Python

```
=(COUNTIFS($I$2:$I$46,"yes",$H$2:$H$46,"yes",$G$2:$G$46,"yes")+COUNTIFS($I$2:$I$46,"yes",$H$2:$H$46,"no",$G$2:$G$46,"no"))/L6



=(COUNTIFS($I$2:$I$46,"yes",$H$2:$H$46,"yes")/L6)*(COUNTIFS($I$2:$I$46,"yes",$G$2:$G$46,"yes")/L6)

+(COUNTIFS($I$2:$I$46,"yes",$H$2:$H$46,"no")/L6)*(COUNTIFS($I$2:$I$46,"yes",$G$2:$G$46,"no")/L6)
```

Git: judge_vs_human.csv (id, split, answer_key, grade, accepted, judge_grounded, judge_relevant, human_grounded, human_relevant); 57 tests.

The AI check

The docstring is the prompt. The retrieval code that comes back runs and looks finished.

1

Docstring

Units, conventions, and what the code must never let through: what you asked for.

2

Code back

Written for the course in the style of an AI answer. Read it.

3

Test first

Plant the case the docstring promises to catch. Run it: it fails.

4

Fix

Find the rule it broke, fix it, run the test again.

Today: the answer key is written into the judge's user message as a reference answer, so the judge scores agreement with the key.

Code that runs is not the same as code that is right.

What you can do now

- Three checks: a key, a citation check, an LLM judge
- A judge: fixed instruction, strict JSON, a pydantic record, temperature 0, no key
- Agreement table and Cohen's kappa; the same words in both columns
- A judge that says yes to everything has kappa 0; a system message is part of the request, so a judge's holds only its own
- Grounded is not right; a verdict is a measurement with an error rate

Week 3, Day 3 Index refresh and corpus drift